11/05 발표 · 정민지 파트 발표용 해설

슬라이드 3~14장 · 장마다 발표 스크립트, 숫자 읽는 법, 예상 질문과 답, 고칠 점

이 문서 사용법

> 11/05 발표의 정민지 파트(3~14장)를 장마다 **한 줄 메시지 → 발표 스크립트 → 숫자 읽는 법 → 예상 질문과 답 → 슬라이드에서 고칠 점** 순서로 정리했습니다.

전체 이야기 흐름 (한 문장)

RQ-VAE로 기사마다 SID를 만들고(3~4장), 그 SID가 기사 내용을 잘 담는지 사람 · 단어 · 임베딩 · 이웃 네 방향으로 확인한 뒤(7~12장), 추천 모델에 넣어 실제로 쓸모 있는지 봤습니다(13~14장, 5~6장).

권장 발표 순서

3 · 4 RQ-VAE → 7 · 8 정성평가 → 9 ~ 12 정량평가 → 13 NRMS →
5 Transformer 탐색 → 6 최종 성능

지금 파일은 3 → 4 → 5 → 6 → 7 … 순서라 결론(6장)이 SID 평가보다 먼저 나옵니다. 위 순서로 바꾸면 “만들고 → 확인하고 → 써 본다”로 이어집니다. 14장은 13장과 겹쳐 생략 가능합니다.

꼭 기억할 숫자

무엇	숫자	장
uniqueness loss로 줄어든 충돌	31.56% → 24.39%	4
사람 판정 “높음” 비율	0% → 27% → 77%	7
본문 TF-IDF 중앙값	0.026 → 0.056 → 0.188	9
원본 임베딩 cosine	0.799 → 0.818 → 0.847 → 0.907	10
자기 복원 평균	0.9275	11
KNN 같은 c1 / c1c2 (무작위 대비)	70.85% / 28.20% (4.5× / 94×)	12
NRMS E5 → SID (유지율)	29.92% → 26.09% (87.2%)	13
Transformer V1 / + level 가중치	27.07% / 29.46%	6
TIGER (비공식 재현)	17.36%	6

공통 범위 · 2~12장의 SID 품질 평가는 RQ-VAE 학습에 쓰인 train 기사 기준입니다(11장은 train + validation, 12장은 validation 보조 결과 있음). 추천 성능(5 · 6 · 13장)은 모두 validation 앞쪽 절반으로 고르고 뒤쪽 절반(test 122,668문제)으로 한 번 잰 숫자입니다.

3장 · RQ-VAE · 어떤 파라미터를 어떻게 탐색했나

RQ-VAE 설계



11/05 발표 슬라이드 3

> RQ-VAE는 **복원 정확도만이 아니라 SID 품질**(충돌 · 사건 일관성 · 코드북 사용 · prefix 의미)까지 함께 보고 골랐습니다.

발표 스크립트 (이렇게 말하면 됩니다)

SID를 만드는 RQ-VAE는 원본 임베딩을 잘 복원한다고 끝이 아닙니다. 서로 다른 기사가 같은 코드를 받으면 추천 모델이 둘을 구분할 수 없기 때문입니다.

그래서 왼쪽 변수들, 즉 코드북 크기(Q2 64~512, Q3 128~1024), 잠재 차원과 hidden 구조, 양자화 방식(STE · Gumbel · Rotation, 정규화 여부), loss 가중치와 학습률 · batch · weight decay를 바꿔 가며 실험했습니다.

판단은 오른쪽 다섯 지표로 했습니다. 복원 loss와 c1c2c3 충돌은 낮을수록, 같은 사건의 c2 일관성, Q2 · Q3 코드 사용률, prefix가 길수록 의미가 가까워지는 정도는 높을수록 좋습니다. 이 탐색의 마지막 weight decay 단계에서 뽑힌 기본 모델이 다음 장의 WD-001입니다.

숫자 읽는 법

지표	방향	뜻
Valid rec loss	↓	validation 기사의 E5 768차원을 얼마나 정확히 복원하나 (MSE)
C123 collision	↓	서로 다른 기사가 같은 (c1, c2, c3)를 받은 비율. 높으면 추천에서 구분 불가
C2 event consistency	↑	전처리에서 같은 event_id를 받은 기사들이 실제로 같은 c2를 받았나
Q2 / Q3 usage	↑	코드북 128 / 512개 중 실제로 쓰인 코드 수. 낮으면 소수 코드에 몰림
ΔC2 / ΔC3	↑	prefix를 한 자리 더 공유할 때 의미 유사도가 얼마나 더 오르나

예상 질문과 답

Q. WD-001은 무슨 뜻인가요?

A. 단계별 RQ-VAE 탐색의 마지막 단계(weight decay)에서 001번 설정입니다. 이후 모든 실험의 기본 RQ-VAE이고, 여기에 uniqueness loss를 더한 것이 UNI입니다.

Q. c1은 왜 학습하지 않고 카테고리로 고정했나요?

A. 설계상 c1은 편집 카테고리 25개로 고정합니다. 사람이 해석할 수 있고 외부 지식(편집 분류)에 맞춰 두려는 의도입니다. 그래서 E5 이웃의 약 29%가 다른 c1을 받는 것은 이 설계의 대가입니다(12장).

Q. 코드북 붕괴(일부 코드만 쓰이는 문제)는 어떻게 막았나요?

A. K-means로 코드북 첫 값을 정해 학습을 시작했고(seed 42 고정), 결과적으로 Q3 512개가 모두 사용됩니다(4장 Q3 사용 코드 512/512).

Q. 최종 checkpoint는 어떻게 골랐나요?

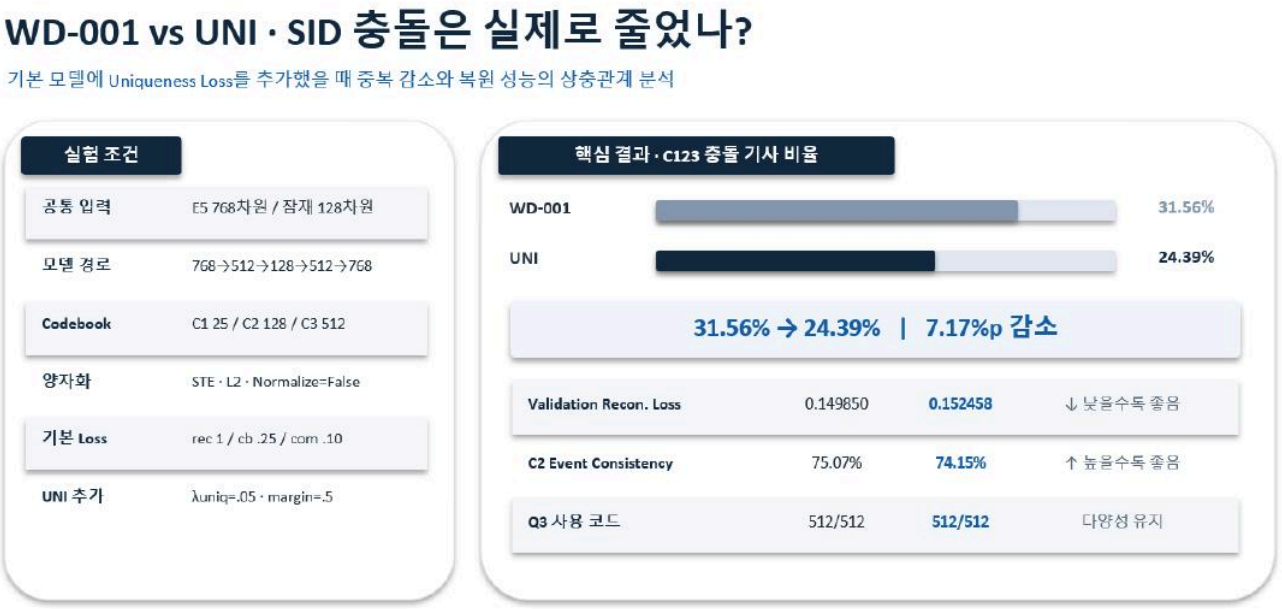
A. 선택이 두 단계입니다. ① 어떤 설정을 쓸지는 오른쪽 다섯 지표를 함께 보고 골랐습니다. ② 각 설정의 학습 run마다, 10 epoch마다 validation 기사 3,122개의 복원 loss를 재서 가장 낮은 epoch(best_rec)을 저장했습니다. 이 규칙은 모든 run에 똑같이 적용했고, UNI는 epoch 180입니다.

슬라이드에서 고칠 점

- 왼쪽 상자의 마지막 줄 “추가 진단 · SimVQ · Temperature · K-means 초기화 · Seed”는 삭제합니다. 주력 탐색 축이 아니고, SimVQ · Gumbel temperature는 최종 모델에 쓰이지 않습니다(최종은 STE, SimVQ 미사용).
- 노트의 “1283 에폭 중 rec loss가 가장 낮았던 체크포인트”: 1283은 설정한 최대 epoch이고 early stopping이 켜져 있었습니다. “validation 복원 loss가 가장 낮은 checkpoint (UNI: epoch 180)”로 쓰는 것이 정확합니다.

4장 · WD-001 vs UNI · SID 충돌은 실제로 줄었나

RQ-VAE 결과



11/05 발표 슬라이드 4

> uniqueness loss를 더하자 **충돌 기사가 31.56% → 24.39%로 7.17%p 줄었고**, 대신 복원과 사건 일관성은 아주 조금 나빠졌습니다(상충관계).

발표 스크립트 (이렇게 말하면 됩니다)

WD-001에서는 기사 세 개 중 하나꼴로 다른 기사와 같은 c1c2c3를 받았습니다.

그래서 같은 (c1, c2, c3)를 받은 기사끼리 latent 유사도가 margin 0.5를 넘으면 벌점을 주는 uniqueness loss(λ 0.05)를 추가했습니다. 나머지 구조와 loss는 그대로입니다.

결과적으로 충돌은 24.39%로 줄었습니다. 대가로 validation 복원 loss는 0.1499에서 0.1525로, 같은 사건 c2 일관성은 75.07%에서 74.15%로 조금 나빠졌습니다. Q3 코드 512개는 여전히 모두 쓰입니다. 이후 실험은 모두 이 UNI SID 기준입니다.

숫자 읽는 법

지표	WD-001	UNI	더 좋은 쪽
c123 충돌 기사 비율 ↓	31.56%	24.39%	UNI (−7.17%p)
Validation 복원 loss ↓	0.149850	0.152458	WD-001 (+0.0026)
C2 event consistency ↑	75.07%	74.15%	WD-001 (−0.92%p)
Q3 사용 코드	512 / 512	512 / 512	같음

충돌 비율은 이렇게 계산합니다

충돌 기사 비율 = c1c2c3가 같은 다른 기사가 있는 기사 수 ÷ 전체 기사 수. 같은 (c1, c2, c3)끼리 묶고, 2개 이상 묶음의 기사를 모두 셉니다. c4는 보지 않습니다(c4는 같은 c1c2c3 안의 순번 = 충돌 표시).

작은 예시 (기사 10개)	묶음	크기
A, B, C	20-122-378	3 ← 충돌
F, G	12-22-288	2 ← 충돌
D / E / H / I / J	각자 다름 (D 21-117-485, E 21-117-147은 c3가 달라 충돌 아님)	1씩
충돌 기사 = A, B, C, F, G = 5개 → 5 ÷ 10		50%

실제 UNI (12,860개)	묶음 수	기사 수
혼자인 묶음	9,723	9,723
2개 / 3개 / 4~17개 묶음	893 / 188 / 148	1,786 / 564 / 787
2개 이상 합계 → 3,137 ÷ 12,860	1,229	24.39%

실제 예: 20-122-378 묶음 = Cecilie 20세(train, c4 0) · Cecilie 21세(train, c4 1) · validation 기사 1개(c4 2) → 세 기사 모두 충돌로 셈(train · validation이 섞여도 셈). 가장 큰 묶음이 17개라 max c4 = 16. 코드에 있는 다른 지표 `c123_collision_rate` = 1 − 묶음 수 ÷ 기사 수 = 14.84%와 섞지 않도록 주의합니다(슬라이드는 기사 비율).

예상 질문과 답

Q. 충돌 24%면 여전히 많지 않나요?

A. c4(같은 c1c2c3 안의 순번)로 기사 구분은 됩니다. 다만 Transformer 점수는 c1~c3만 쓰므로 같은 c1c2c3 기사는 같은 점수를 받습니다. 그래서 1P4N 후보를 만들 때 정답과 c1c2c3가 같은 오답은 뺐습니다. 더 줄이려면 λ를 키우거나 c3를 늘려야 하는데, 복원 · 일관성과의 상충이 커집니다.

Q. c2 사건 일관성 74%는 무슨 뜻인가요?

A. 같은 사건 기사들 안에서 가장 많이 나온 c2의 비율 평균입니다. UNI는 학습 때 c2를 기사마다 따로 고르기 때문에 같은 사건이라도 c2가 갈릴 수 있습니다. 원래 설계는 사건 단위 c2(일치율 100%)입니다.

Q. uniqueness loss는 어떻게 계산하나요?

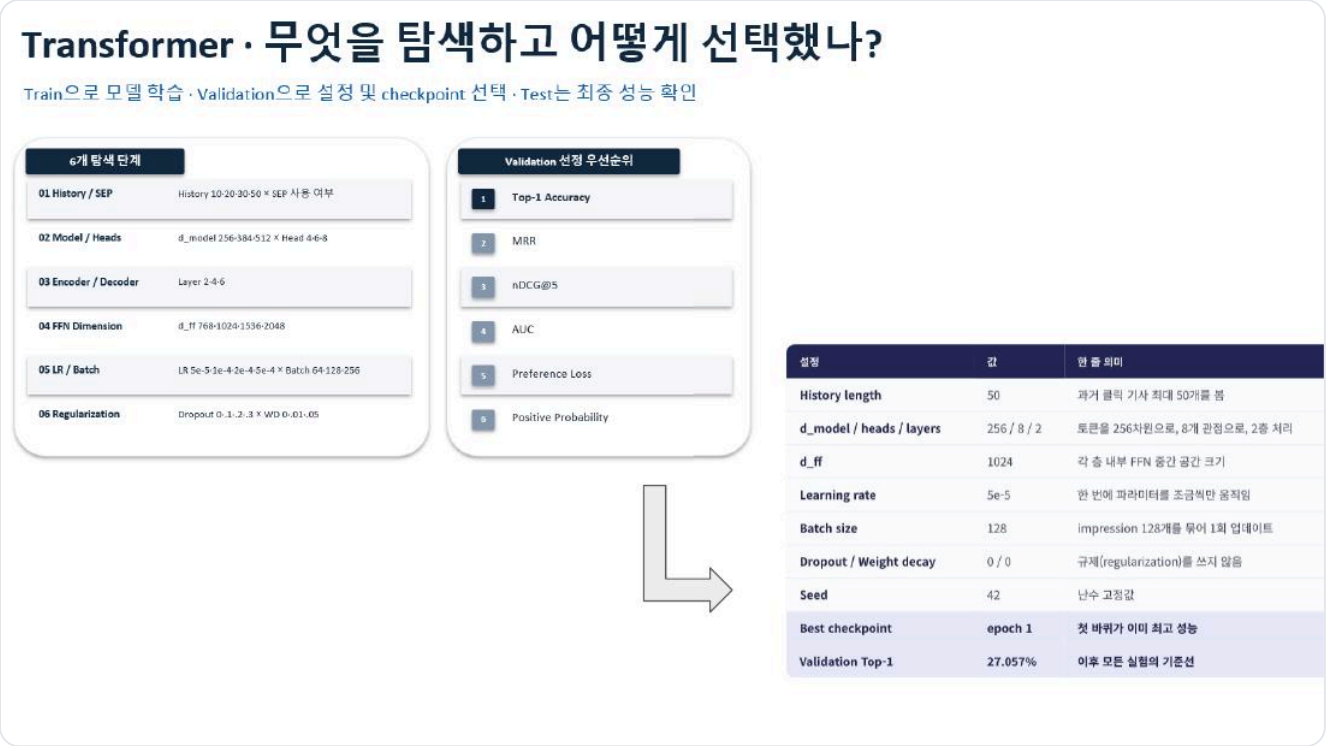
A. 같은 (c1, c2, c3)에 들어간 기사 쌍의 latent cosine이 0.5를 넘는 만큼을 벌점으로 더합니다. 너무 비슷하게 뭉친 기사들을 서로 밀어내 다른 코드를 받게 합니다.

슬라이드에서 고칠 점

- 지표마다 기준 데이터가 다릅니다: 복원 loss는 validation 기사 3,122개, c123 충돌 · C2 사건 일관성은 train + validation 전체 12,860개. 표 아래 한 줄로 적어 두면 좋습니다(UNI 충돌을 나눠 보면 train끼리 20.92%, validation끼리 14.38%).
- UNI 열이 모두 파란 굵은 글씨라 "전부 좋아졌다"로 읽힙니다. 충돌만 개선, 복원 · 일관성은 소폭 악화이므로 색을 나눠 표시하는 게 좋습니다.

5장 · Transformer · 무엇을 탐색하고 어떻게 선택했나

Transformer 설계



11/05 발표 슬라이드 5

> 96,768가지 조합을 **6단계 · 84 run**으로 줄여 validation으로만 골랐고, 최종 V1은 256 / 8 head / 2층, history 50, lr 5e-5, 규제 없음입니다.

발표 스크립트 (이렇게 말하면 됩니다)

조합이 96,768가지라 전부 돌릴 수 없어서, 서로 얽힌 파라미터끼리 묶어 6단계로 나눴습니다. 단계마다 상위 2개 설정만 다음 단계로 넘겨 총 84 run을 돌렸습니다.

선택은 validation 지표로만 했습니다. Top-1 → MRR → nDCG@5 → AUC → Preference Loss → Positive Probability 순서로, 앞 지표가 허용 범위(Top-1은 0.5%p) 안에서 같으면 다음 지표로 비교했습니다. 마지막 상위 3개는 seed 42 · 123 · 2026으로 다시 확인했습니다.

최종 설정은 history 50 · SEP 없음, d_model 256 · head 8 · 2층, d_ff 1024, lr 5e-5 · batch 128, dropout · weight decay 0입니다. validation Top-1 27.06%이고, best checkpoint가 epoch 1입니다. 첫 epoch 뒤로는 train만 좋아지고 validation은 떨어지는 빠른 과적합입니다.

숫자 읽는 법

단계	함께 탐색	후보	run 수	선정
1	History × SEP	10 · 20 · 30 · 50 × 사용 여부	8	50 / 미사용
2	d_model × heads	256 · 384 · 512 × 4 · 6 · 8 (나누어떨어지는 7쌍)	14	256 / 8
3	layers	2 · 4 · 6	6	2
4	d_ff	768 · 1024 · 1536 · 2048	8	1024
5	LR × batch	5e-5 · 1e-4 · 2e-4 · 5e-4 × 64 · 128 · 256	24	5e-5 / 128
6	dropout × weight decay	0 · 0.1 · 0.2 · 0.3 × 0 · 0.01 · 0.05	24	0 / 0
합계			84	validation Top-1 27.06%

예상 질문과 답

Q. 왜 grid search를 다 하지 않았나요?

A. 96,768 조합은 불가능해서, 같이 움직이는 파라미터(예: lr과 batch)는 같은 단계에서 조합으로 보고 나머지는 단계별로 나눴습니다. 단계마다 2개를 남겨 한 번에 하나만 고를 때보다 잘못된 길로 빠질 위험을 줄였습니다.

Q. best epoch가 1이면 학습이 안 된 것 아닌가요?

A. 첫 epoch에서 이미 validation 최고가 나오고, 이후에는 train loss만 내려갑니다. test 정답의 98% 이상이 train에서 본 적 없는 새 기사라 오래 학습할수록 train 기사에 맞춰지는 과적합입니다.

Q. 선택 기준이 Top-1 하나인가요?

A. 두 가지 선택을 구분해야 합니다. ① 어떤 설정을 쓸지는 Top-1 → MRR → nDCG@5 → AUC → Preference Loss → Positive Probability 순서로, 각 지표의 허용 오차 안이면 다음 지표로 비교했습니다(그래도 같으면 파라미터 수 → epoch 시간 → GPU 메모리). ② 한 설정의 학습 중 몇 번째 epoch을 쓸지는 validation Top-1 하나로 골랐습니다(최대 30, patience 5).

Q. test로 고른 것은 없나요?

A. 없습니다. 모든 선택은 validation 앞쪽 절반으로 했고 test는 최종 설정을 정한 뒤 한 번만 쟀습니다.

Q. 27.06%와 test 27.07%가 다른 데이터인가요?

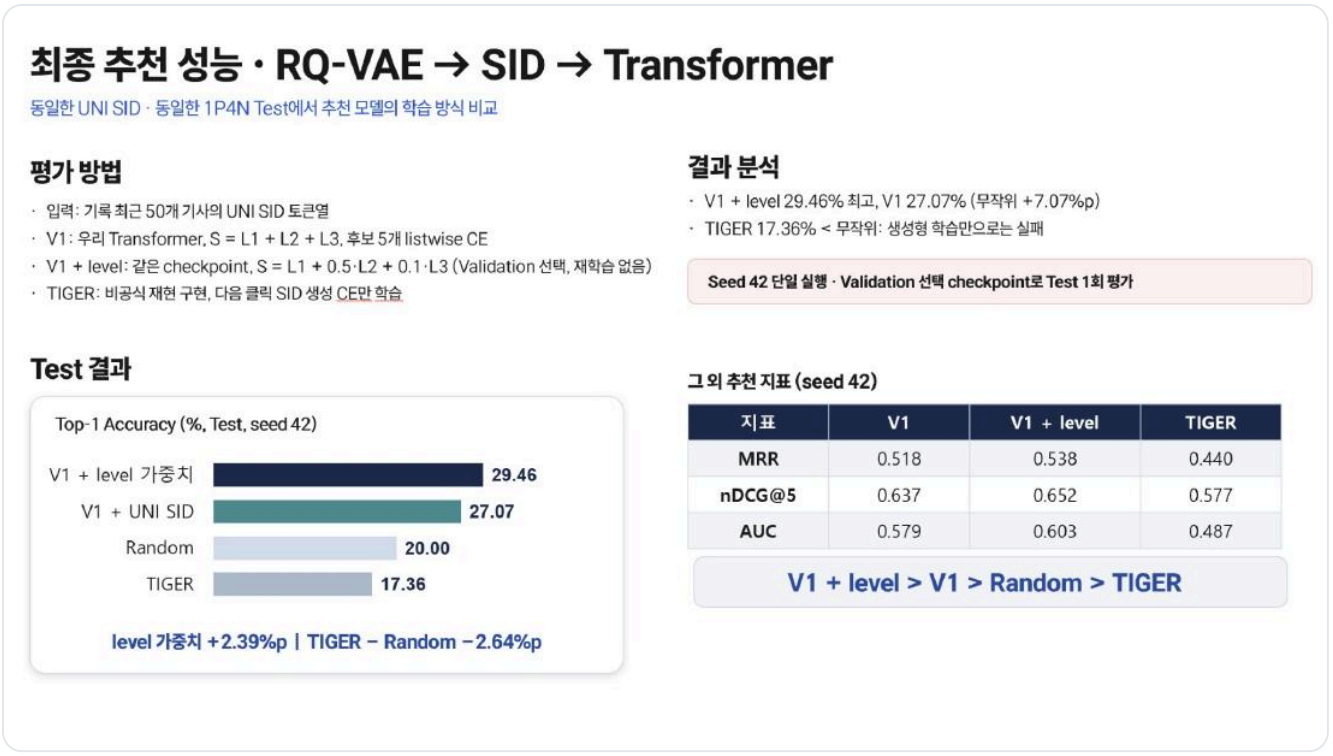
A. 탐색은 후보 순서를 섞기 전 데이터(정답이 항상 0번 자리)에서 했고, 최종 설정은 순서를 섞은 데이터로 다시 학습했습니다. 섞은 데이터 기준 validation 27.06%, test 27.07%입니다.

슬라이드에서 고칠 점

- 오른쪽 최종 설정 표가 슬라이드 오른쪽 밖으로 잘립니다.
- “단계마다 상위 2개 → 총 84 run, 마지막 상위 3개 seed 확인”을 한 줄 넣으면 탐색 방식이 바로 이해됩니다.

6장 · 최종 추천 성능 · RQ-VAE → SID → Transformer

최종 결과 Test



11/05 발표 슬라이드 6

> 같은 UNI SID · 같은 test에서 **V1 27.07%, level 가중치 29.46%**로 무작위(20%)보다 확실히 높고, 생성형 TIGER는 17.36%로 무작위보다 낮습니다.

발표 스크립트 (이렇게 말하면 됩니다)

마지막으로 RQ-VAE가 만든 SID를 Transformer 추천까지 통과시킨 결과입니다. 세 모델 모두 같은 UNI SID, 같은 test 122,668문제입니다.

V1은 기록 기사들의 SID 토큰을 읽고 후보마다 c1 · c2 · c3 확률을 내서, 그 log 확률 합 S = L1 + L2 + L3로 후보 5개를 채점합니다. 학습은 정답 후보의 점수가 오답보다 높아지도록 합니다. 결과는 27.07%입니다.

같은 모델에서 점수식만 S = L1 + 0.5·L2 + 0.1·L3로 바꾸면 29.46%가 됩니다. 가중치는 validation에서 골랐고 재학습은 하지 않았습니다. c3를 약하게 반영할수록 좋아지는 이유는, c3가 기사 수준 코드라 새 기사에 잘 일반화되지 않기 때문입니다.

TIGER는 같은 점수식을 쓰지만 다음 클릭 SID를 생성하는 것만 학습합니다. 17.36%로 무작위보다 낮습니다. test 정답 중 train에서 클릭된 적 있는 기사는 1.7%인데 오답은 12.2%라, 학습 때 본 기사에 확률을 몰아주는 생성형 학습은 오답을 더 높게 봅니다.

숫자 읽는 법

모델	Top-1	MRR	nDCG@5	AUC
V1 + level 가중치	29.46%	0.538	0.652	0.603
V1 + UNI SID	27.07%	0.518	0.637	0.579
Random	20.00%	—	—	0.500
TIGER (비공식 재현 구현)	17.36%	0.440	0.577	0.487

실험 방법 상세 — train · validation · test · epoch · seed

RQ-VAE (UNI, epoch 180) → 기사 12,860개 SID → train으로 모델 학습 →

validation half로 best epoch · level 가중치 선택 → test로 딱 한 번 채점

묶음	문제 수	어디서	쓴 곳
train	V1 232,874 (1P4N) TIGER 클릭 233,161	EB-NeRD train 주 클릭 기록	모델 학습
validation half	122,549	validation 주 앞쪽 절반 (시간순)	best epoch 선택, level 가중치 선택
test	122,668	validation 주 뒤쪽 절반	성적 보고 (한 번만)

세 모델의 test는 같은 문제입니다(노출 · 후보 5개 · 정답 일치를 비교 스크립트로 확인). 1P4N = 클릭한 기사 1개 + 보고도 안 클릭한 기사 4개, 후보 순서는 섞음, 무작위 Top-1 20%.

	① V1	② V1 + level 가중치	③ TIGER
정체	우리 팀 Transformer	V1과 같은 모델, 점수식만 바꿈	비공식 공개 재현 구현 (EdoardoBotta, 수정 없음)
train 데이터	1P4N 232,874문제 (정답 + 오답 4)	학습 안 함	1P4N 이전 기록의 클릭 233,161개 (오답 없음)
학습 목표	후보 5개 점수에 listwise CE (정답이 오답보다 높게)	—	다음 클릭 SID를 한 자리씩 맞히기 (token CE)
기록 입력	최근 50개 기사 (c1 c2 c3 c4)	—	최근 20개 기사 (c1 c2 c3)
모델	d256 · 8 head · 2층 · d_ff 1024 · dropout 0	—	d384 · 6 head · 4층 · d_ff 1024 · dropout 0.1
최적화	AdamW lr 5e-5 · wd 0 · batch 128	—	lr 1e-3 · batch 640
최대 epoch	30 (5번 연속 개선 없으면 정지)	—	30 (같은 규칙 → epoch 6에서 정지)
best epoch 기준	validation Top-1	—	validation Top-1
best epoch	1	V1 checkpoint 그대로	1
seed	42	42 (V1과 같음)	42
후보 점수	S = L1 + L2 + L3	S = 1·L1 + 0.5·L2 + 0.1·L3	S = L1 + L2 + L3

L1 = log P(c1), L2 = log P(c2 | c1), L3 = log P(c3 | c1, c2). 후보마다 S를 계산해 가장 큰 후보를 추천합니다.

level 가중치를 고른 방법

1. V1 checkpoint로 validation half와 test 문제마다 L1 · L2 · L3를 계산해 저장합니다.
2. α2 7개 {0, 0.25, 0.5, 0.75, 1.0, 1.25, 1.5} × α3 6개 {0, 0.1, 0.25, 0.5, 0.75, 1.0} = 42가지를 **validation half**에서 비교합니다 (α1 = 1 고정).
3. 가장 좋은 (1, 0.5, 0.1)을 골라 **test에 한 번** 적용합니다. 재학습은 없습니다.

best epoch이 1인 이유

TIGER epoch	train loss	val Top-1	val AUC
0 (학습 전)	—	20.00%	0.502
1 (best)	7.359	16.38%	0.458
2	6.714	15.53%	0.452
3 ~ 6	6.63 ~ 6.67	15.0 ~ 15.7%	0.441 ~ 0.446

- V1: 첫 epoch에서 validation Top-1이 최고(27.06%)였고, 이후에는 train loss만 내려가고 validation은 떨어졌습니다.
- TIGER: train loss는 내려가는데 validation은 epoch 1부터 무작위보다 낮고 계속 나빠져 epoch 6에서 멈췄습니다.
- 공통 원인: test 정답의 98% 이상이 train에서 클릭된 적 없는 새 기사라, 오래 학습할수록 train 기사에 맞춰집니다. test 정답 중 train에서 클릭된 기사는 1.7%, 오답은 12.2%라 생성형 학습은 오답을 더 높게 봅니다. TIGER도 c1, c1 + c2까지만 쓰면 23.0%, 23.7%로 무작위보다 낮고, c3를 더하면 17.4%로 떨어집니다.

주의 · 세 모델 모두 seed 42 한 번이라 1%p 안쪽 차이는 같은 수준으로 봅니다(V1 대 TIGER −9.7%p, level 가중치 +2.4%p는 그보다 큼). TIGER는 공개 구현 설정 그대로이고 우리 데이터에 맞춘 탐색은 하지 않았습니다. 근거: `experiments/uni_v1_test_eval/` , `experiments/uni_tiger_reference_v1/results_aws/summary_seed42.md`

예상 질문과 답

Q. TIGER는 논문 저자 코드인가요?

A. 아닙니다. TIGER(Rajput et al., NeurIPS 2023) 저자는 코드를 공개하지 않았고, EdoardoBotta/RQ-VAE-Recommender는 비공식 PyTorch 재현 구현입니다. 우리 RQ-VAE 코드의 원본 저장소이기도 합니다. 수정 없이 썼습니다.

Q. TIGER 방법이 틀렸다는 뜻인가요?

A. 아닙니다. 논문은 같은 상품이 반복해서 나오는 Amazon 데이터에서 전체 아이템을 생성해 Recall@K로 평가했습니다. 우리는 새 기사 위주의 뉴스에서 후보 5개를 비교하는 다른 과제에 그대로 썼습니다.

Q. level 가중치를 test 보고 고른 건 아닌가요?

A. 아닙니다. validation 앞쪽 절반에서 격자로 골랐고, test에서도 +2.39%p가 유지됐습니다.

Q. seed가 하나인데 믿을 수 있나요?

A. V1 · TIGER는 seed 42 한 번이라 1%p 안쪽 차이는 같은 수준으로 봐야 합니다. V1 대 TIGER(−9.7%p)나 level 가중치(+2.4%p)는 그보다 큽니다.

Q. NRMS(13장)와 비교하면요?

A. NRMS + SID 26.09%, NRMS + E5 29.92%(3-seed)입니다. 모델이 달라 직접 비교는 아니지만, V1 + level 가중치(29.46%)가 E5를 쓴 NRMS 수준에 가깝습니다.

슬라이드에서 고칠 점

- 발표 흐름상 결론 장이라 SID 평가(7~12장)와 NRMS(13장) 뒤로 옮기는 게 자연스럽습니다.

7장 · 정성평가 · 평가 설계와 판정 기준

SID 품질 정성



11/05 발표 슬라이드 7

> 사람이 기사를 직접 읽어 보니 SID를 더 공유할수록 내용이 비슷했습니다: “높음” 비율 **0% → 27% → 77%**.

발표 스크립트 (이렇게 말하면 됩니다)

숫자 지표가 놓칠 수 있는 “실제로 읽었을 때 비슷한가”를 확인했습니다. 같은 사건인지가 아니라 기사 전체 내용이 얼마나 비슷한지를 봤습니다.

표본은 쌍이 아니라 그룹을 먼저 뽑았습니다. Prefix-3(c1c2c3 같음)는 크기별로 2~3개짜리 15그룹, 4~5개 10그룹, 6~8개 5그룹, 모두 30그룹입니다. Prefix-2(c1c2 같고 c3 다름)는 기사 4~10개 그룹 15개입니다. 대조군 Different는 SID가 전부 다른 30쌍입니다. 그룹의 기사는 전부 읽었습니다.

판정은 높음 · 중간 · 낮음 3단계입니다. AI가 1차 판정하고 사람이 교차 검토했습니다. 결과는 Different 30쌍 모두 낮음, Prefix-2는 4 · 5 · 6으로 섞임, Prefix-3는 30그룹 중 23개가 높음입니다.

숫자 읽는 법

조건	높음	중간	낮음	"높음" 비율
Different (30쌍)	0	0	30	0%
Prefix-2 (15그룹)	4	5	6	27%
Prefix-3 (30그룹)	23	5	2	77%

예상 질문과 답

Q. 누가 판정했나요?

A. AI가 덴마크어 원문(제목 · 부제 · 본문)을 읽고 1차 판정한 뒤 사람이 교차 검토해 확정했습니다. 예를 들어 P3-04는 AI 1차 "중간"을 사람이 "낮음"으로 바꿨습니다. 판정자 간 일치도는 따로 계산하지 않았습니다.

Q. 77%를 전체 SID에 일반화할 수 있나요?

A. 없습니다. 큰 그룹에서 의미가 무너지는지 보려고 큰 그룹을 일부러 많이 뽑았습니다(실제로는 2~3개짜리 그룹이 대부분). 비율 추정용이 아니라 진단용 표본입니다.

Q. c2 그룹이 128개짜리도 있는데 그건 읽었나요?

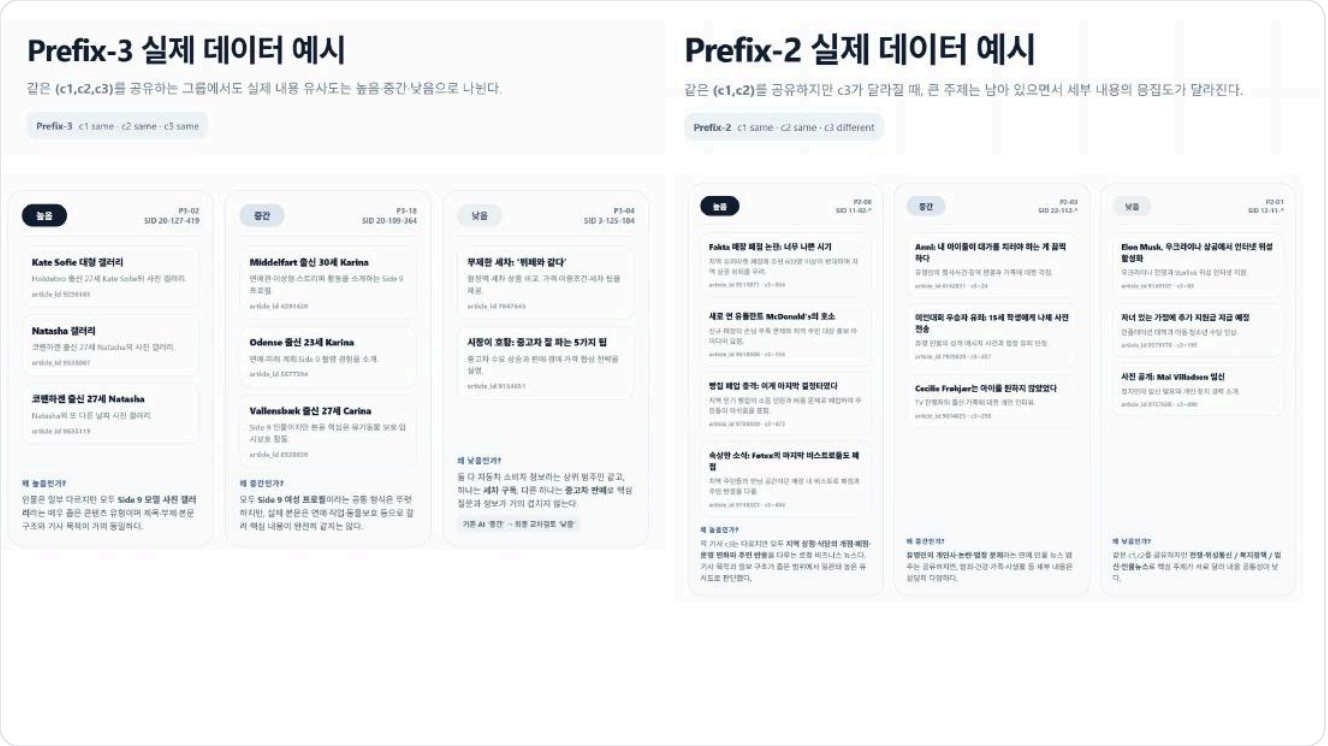
A. 아닙니다. Prefix-2는 기사 4~10개 그룹만 대상이었습니다. 큰 c2 그룹이 주제로 묶였는지는 5 · 6장 정량 지표가 보완합니다.

슬라이드에서 고칠 점

- 발표자 노트가 9장 상자그림 설명이 잘못 복사된 것입니다. 표본 설계와 판정 방법으로 바꾸는 것이 좋습니다.
- 슬라이드에 표본 설계(30그룹 · 15그룹 · 30쌍)와 "AI 1차 → 사람 검토"가 없어 숫자의 근거가 안 보입니다.

8장 · Prefix-3 · Prefix-2 실제 데이터 예시

SID 품질 정성



11/05 발표 슬라이드 8

> 같은 c1c2c3 안에서도 높음 · 중간 · 낮음이 갈리고, **c3가 다르면(Prefix-2) 큰 주제만 남고 세부가 흩어집니다.**

발표 스크립트 (이렇게 말하면 됩니다)

왼쪽은 c1c2c3가 같은 그룹입니다. 높음(P3-02)은 인물은 달라도 모두 Side 9 사진 갤러리라 형식 · 목적이 거의 같습니다. 중간(P3-18)은 Side 9 여성 프로필이라는 형식은 같지만 본문 내용이 연예 · 직업 · 동물보호로 갈립니다. 낮음(P3-04)은 "자동차 소비자 정보"라는 큰 범주만 같고 세차 구독과 중고차 판매라 정보가 거의 겹치지 않습니다.

오른쪽은 c1c2만 같고 c3가 다른 그룹입니다. 높음(P2-08)은 지역 상점 · 식당의 개점 · 폐점 뉴스로 일관됩니다. 중간(P2-03)은 유명인의 사생활 · 법정 문제라는 범주는 같지만 세부 주제가 다양합니다. 낮음(P2-01)은 머스크 위성 · 자녀수당 · 연예인 임신으로 서로 다릅니다.

즉 c2까지 같으면 큰 주제가, c3까지 같으면 기사 유형과 내용까지 비슷해지는 경향이 사람 눈으로도 보입니다.

숫자 읽는 법

그룹	SID	판정	이유
P3-02	20-127-419	높음	Side 9 사진 갤러리, 형식 · 목적 거의 동일
P3-18	20-109-364	중간	프로필 형식은 같고 본문 핵심은 다름
P3-04	3-125-184	낮음	자동차 소비자 정보라는 큰 범주만 같음
P2-08	11-92-*	높음	지역 상점 개점 · 폐점 뉴스
P2-03	22-112-*	중간	유명인 사생활 · 법정, 세부 주제 다양
P2-01	12-11-*	낮음	위성 · 복지정책 · 연예로 핵심 주제가 다름

예상 질문과 답

Q. 왜 형식이 같은 기사가 같은 SID로 묶이나요?

A. SID 입력 임베딩이 제목 + 부제로만 만들어집니다. Side 9처럼 제목 틀이 같은 템플릿 기사는 본문이 달라도 비슷해 보입니다. 설계상 c3는 “어떻게 서술했나”를 담는 자리라 형식 묶음 자체는 의도와 맞지만, 내용 차이는 볼 수 없습니다.

Q. 예시는 어떻게 골랐나요?

A. 7장에서 판정한 그룹 중 판정 단계마다 대표 하나씩 고른 설명용 예시입니다. 전체 분포는 7장 표와 9장 정량이 보여 줍니다.

슬라이드에서 고칠 점

- 카드 안 글씨가 작아 발표 화면에서 읽기 어렵습니다. 그룹당 기사 2개 정도로 줄이거나 Prefix-3와 Prefix-2를 두 장으로 나누는 것을 권합니다.

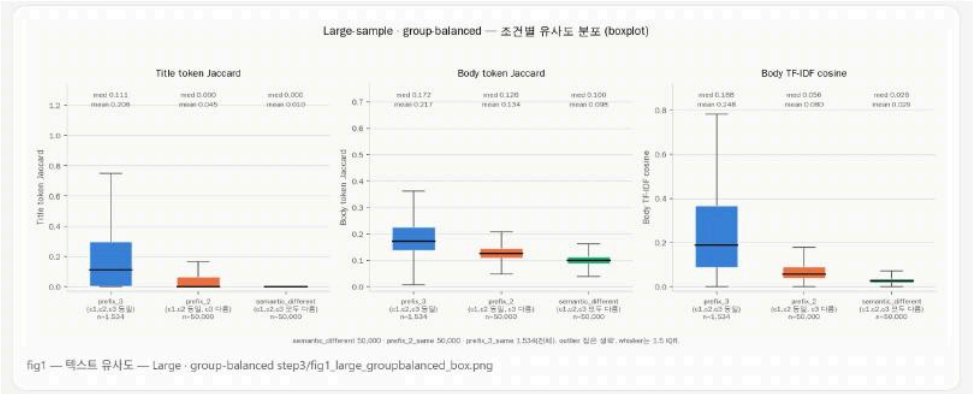
9장 · 정량평가1 · Jaccard / TF-IDF

SID 품질 정량

정량평가 **1 • Jaccard/TF-IDF**

같은 **preifx-3 group** 에 기사 **A,B,C 3** 개가 있다면 **A-B/A-C/B-C** 에 대해 수행 **(prefix-3 : 1,534 pair** 전수 **/prefix-2 : 128,164** 개 중 **50,000** 개 샘플링 **/diff: 36,384,698** 개 중 **50,000** 개 샘플링 **)**

* 기사 임베딩은 **title+subtitle(abstract)** 로 생성



평균

Different	Prefix-2	Prefix-3
0.0291	**0.0799**	**0.2484**
baseline	c1,c2 공유	c1,c2,c3 공유

결과 분석

SID 를 더 많이 공유할수록 실제 본문 TF-IDF 유사도가 높아짐 . median 도 같은 방향이므로 소수의 높은 pair 가 평균만 끌어올린 결과도 아님

11/05 발표 슬라이드 9

> SID를 더 공유할수록 **모델이 보지 않은 본문**도 더 비슷합니다. Body TF-IDF cosine 중앙값 **0.026 → 0.056 → 0.188**.

발표 스크립트 (이렇게 말하면 됩니다)

정성평가는 표본이 작아서, 같은 질문을 큰 규모의 숫자로 확인했습니다. 대상은 본문이 있는 train 기사 9,338개입니다.

쌍은 세 종류입니다. Prefix-3는 가능한 1,534쌍 전부, Prefix-2는 128,164쌍 중 5만 쌍, Different는 약 3,638만 쌍 중 5만 쌍을 그룹 균형으로 뽑았습니다. 같은 쌍마다 지표 세 개를 계산했습니다.

Title Jaccard는 제목 단어가 겹치는 비율, Body Jaccard는 본문 단어가 겹치는 비율, Body TF-IDF cosine은 본문에서 흔한 단어는 낮게, 드문 특징 단어는 높게 쳐서 비교한 값입니다.

SID는 제목과 부제만 보고 만들었는데도 본문 지표가 Different < Prefix-2 < Prefix-3 순서로 올라갑니다. 평균만이 아니라 중앙값도 같은 방향이라, 일부 높은 쌍이 평균을 끌어올린 결과가 아닙니다.

숫자 읽는 법

지표	Different	Prefix-2	Prefix-3
Title Jaccard 중앙값 (평균)	0.000 (0.010)	0.000 (0.045)	0.111 (0.206)
Body Jaccard 중앙값 (평균)	0.100 (0.098)	0.126 (0.134)	0.172 (0.217)
Body TF-IDF 중앙값 (평균)	0.026 (0.029)	0.056 (0.080)	0.188 (0.248)
TF-IDF 0.05 미만 쌍	88%	43%	12%

예상 질문과 답

Q. TF-IDF cosine은 어떻게 계산하나요?

A. 기사마다 단어별 점수 = (1 + ln 등장 횟수) × IDF(드문 단어일수록 큼)를 매긴 벡터를 만들고, 길이를 1로 맞춘 뒤 두 기사에 함께 있는 단어끼리 곱해 더합니다. IDF는 9,338개 본문으로 한 번 계산했고, 60% 이상 기사에 나오는 흔한 단어와 1개 기사에만 나오는 단어는 뺐습니다.

Q. Jaccard는 stopword를 안 뺐나요?

A. 안 뺐습니다. 그래서 Jaccard는 흔한 단어 겹침이 섞인 보조 지표로 쓰고, 핵심은 흔한 단어를 깎는 TF-IDF입니다.

Q. 쌍 수가 다른데 비교해도 되나요?

A. 세 조건을 모두 1,534쌍으로 맞춘 비교도 했고 순서가 같았습니다. 그룹 균형 대신 무작위로 쌍을 뽑아도 결론은 같았습니다.

Q. 0.188이면 높은 건가요?

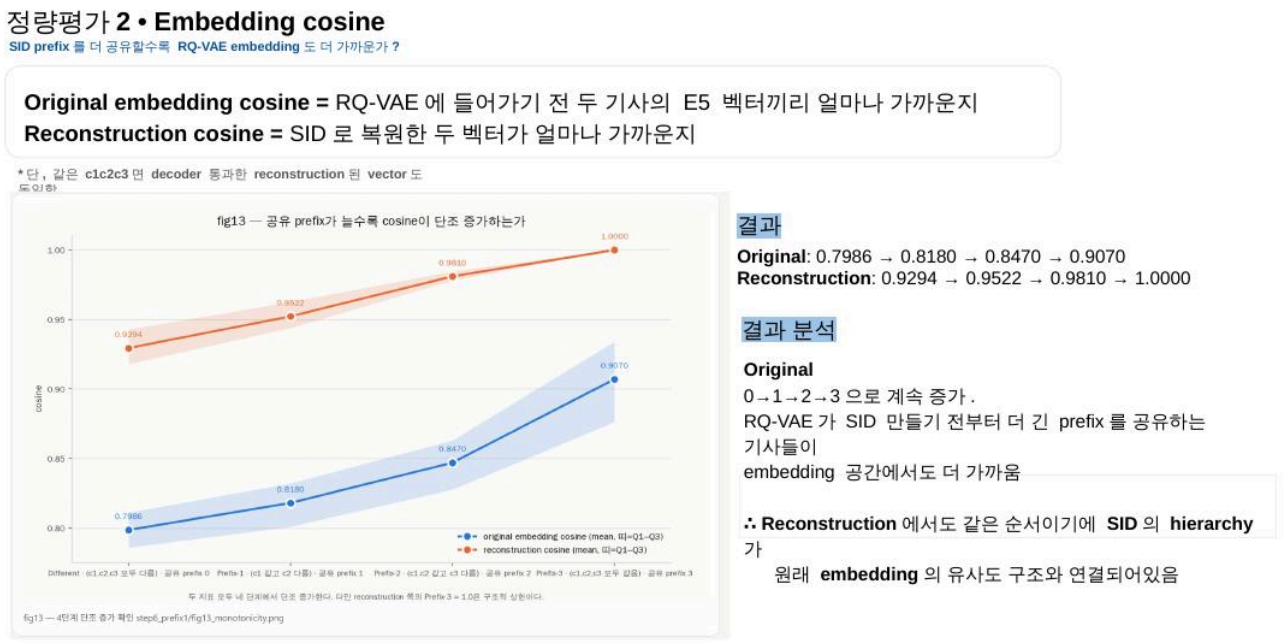
A. 본문 TF-IDF는 원래 값이 낮게 나오는 지표라 절대값보다 기준선(Different 0.026)과 비교합니다. Prefix-3 중앙값은 약 7배이고, 상자(가운데 50%)가 Different 상자와 겹치지 않습니다. 다만 Prefix-3에도 0.05 미만 쌍이 12% 있습니다.

슬라이드에서 고칠 점

- 오타: “preifx-3” → “prefix-3”.
- 아래 카드의 “평균”이 어떤 지표인지 없습니다. “Body TF-IDF cosine 평균”으로 적어야 합니다.
- 대상(train 중 본문 있는 기사 9,338개)을 부제에 넣으면 좋습니다.

10장 · 정량평가2 · Embedding cosine

SID 품질 정량



11/05 발표 슬라이드 10

> 공유 prefix가 0 → 1 → 2 → 3으로 늘수록 **원본 임베딩도, 복원 임베딩도 단조 증가**합니다. SID 계층이 원래 임베딩의 거리 구조를 따릅니다.

발표 스크립트 (이렇게 말하면 됩니다)

9장이 단어 수준이었다면, 여기는 RQ-VAE가 보는 임베딩 공간입니다. 9장과 같은 쌍에 Prefix-1(c1만 같고 c2 다름) 5만 쌍을 더해 네 단계로 봤습니다.

파란 선은 RQ-VAE에 넣기 전 두 기사의 E5 벡터 cosine, 주황 선은 각 기사의 SID로 복원한 벡터끼리의 cosine입니다. 점은 평균, 띠는 가운데 50% 쌍의 범위입니다.

파란 선이 0.7986 → 0.8180 → 0.8470 → 0.9070으로 계속 오릅니다. 긴 prefix를 공유하게 될 기사들은 원래 임베딩에서도 가까웠다는 뜻입니다. 주황 선도 같은 순서라 SID만 남겨도 그 구조가 유지됩니다.

주황 선이 전부 높은 것은 코드 3개로 압축했다 펴면서 벡터들이 서로 가까워진 효과입니다. Prefix-3의 1.0은 decoder 입력이 같아서 자동으로 나오는 값이라 근거에서 뺍니다.

숫자 읽는 법

	Different	Prefix-1	Prefix-2	Prefix-3
Original cosine (평균)	0.7986	0.8180	0.8470	0.9070
Reconstruction cosine (평균)	0.9294	0.9522	0.9810	1.0000 (구조상)

예상 질문과 답

Q. Different인데도 0.80이면 높은 것 아닌가요?

A. E5는 contrastive 학습을 낮은 temperature(0.01)로 해서 모든 문장 벡터가 좁은 방향에 몰려 있습니다. 무관한 기사끼리도 0.77~0.80이 나오는 척도라 절대값이 아니라 단계 간 증가를 봅니다. Prefix-3는 바닥(0.80)에서 1까지 여유의 절반 이상(+0.108)을 올라간 셈입니다.

Q. 파란 선과 주황 선은 같은 쌍인가요?

A. 네. 같은 쌍에서 비교하는 벡터만 다릅니다(원본 E5 vs SID 복원). 쌍은 9장과 같고 Prefix-1만 새로 추가했습니다.

Q. 0.93이면 93% 복원인가요?

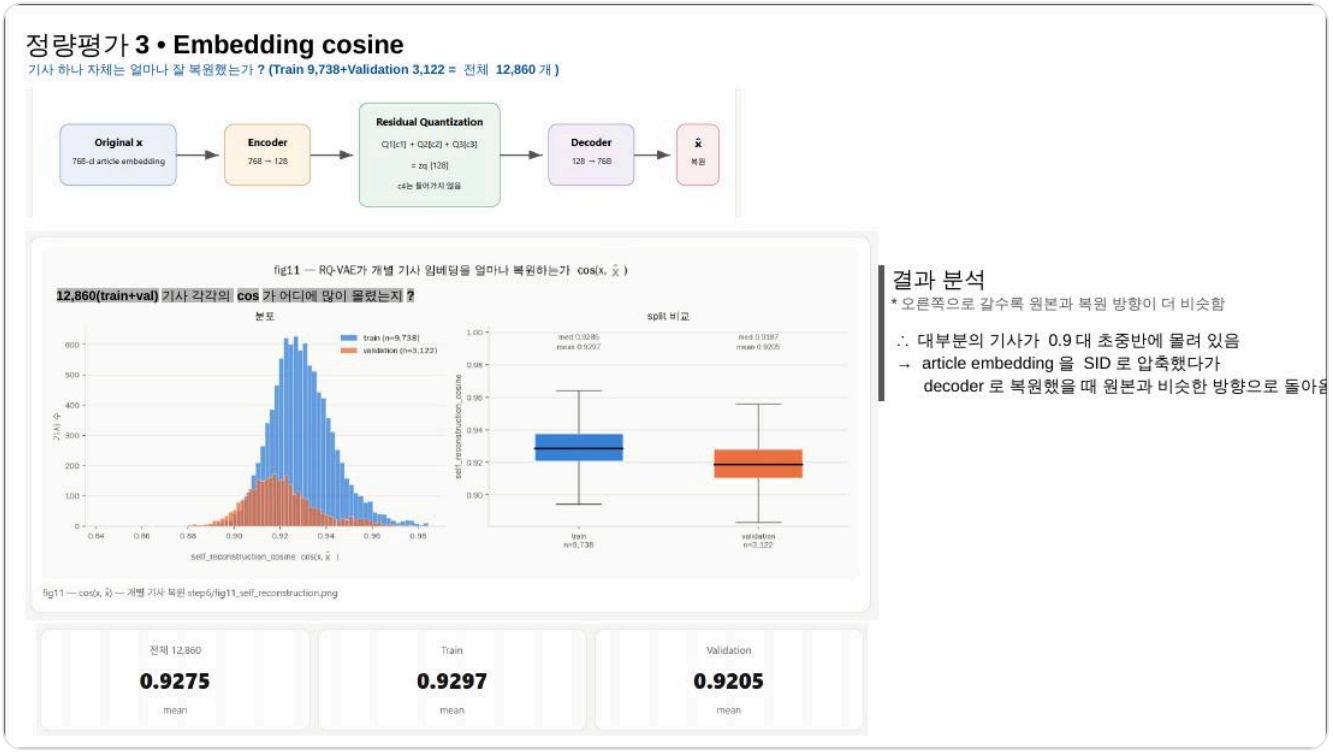
A. 아닙니다. cosine은 퍼센트가 아닙니다. 복원 공간은 압축으로 거리가 좁혀져 무관한 쌍도 0.93이 나옵니다.

슬라이드에서 고칠 점

- 왼쪽 위 각주 "*단, 같은 c1c2c3면 … 동일함"이 줄이 넘어가 그래프와 겹칩니다.
- Prefix-1의 정의(c1 같고 c2 다름)가 슬라이드에 없습니다. 그래프 아래 x축 설명만으로는 작게 보입니다.

11장 · 정량평가3 · 기사 하나의 자기 복원

SID 품질 정량



11/05 발표 슬라이드 11

> 기사 하나를 SID 세 자리로 압축했다가 되살려도 원본과 같은 방향으로 돌아옵니다. 평균 **0.9275**, 가장 낮은 기사도 0.842.

발표 스크립트 (이렇게 말하면 됩니다)

5 · 6장이 기사 쌍이었다면 여기는 기사 하나입니다. 원본 E5를 encoder로 128차원으로 줄이고, SID 세 자리의 코드북 벡터를 더한 뒤 decoder로 다시 768차원으로 펴서 원본과의 cosine을 계산했습니다. c4는 decoder에 들어가지 않습니다.

대상은 추천 기록에 실제로 쓰인 기사 12,860개 전부(train 9,738 + validation 3,122)입니다. 쌍을 만들지 않으니 둘을 함께 볼 수 있고, 학습 때 본 기사(train)와 처음 보는 기사(validation)를 나눠 비교했습니다.

대부분 기사가 0.92~0.94에 몰려 있고 크게 실패한 기사는 없습니다. validation은 0.9205로 train(0.9297)보다 조금 낮지만 차이가 작아, 새 기사에도 SID가 잘 붙는다고 볼 수 있습니다.

숫자 읽는 법

	전체 12,860	Train 9,738	Validation 3,122
평균	0.9275	0.9297	0.9205
중앙값	0.926	0.9285	0.9187
하위 25% / 상위 25%	0.918 / 0.936	—	—
최저 / 최고	0.842 (Kujoneri) / 0.984 (Sofie)	둘 다 train 기사	

예상 질문과 답

Q. 이 점수가 좋은 건가요? 비교 기준이 있나요?

A. 모든 기사에 같은 "전체 평균 벡터"를 넣어도 0.895, 같은 카테고리 평균은 0.904, 같은 c1c2 평균은 0.914가 나옵니다. SID 복원 0.928은 이보다 높아 SID가 기사별 정보를 담고 있지만, 차이는 크지 않습니다(E5의 공통 방향 때문에 바닥이 높음).

Q. 왜 validation이 조금 낮나요?

A. 학습 때 보지 않은 기사이고, validation 기사는 자기에게 가장 가까운 c2가 아니라 사건 코드(사건 기사 평균으로 정한 c2)를 물려받기 때문입니다.

Q. validation 숫자는 믿을 만한가요?

A. RQ-VAE checkpoint를 같은 validation 기사의 복원 loss로 골랐기 때문에 약간 낙관적일 수 있습니다. 고른 것이 epoch 하나라 영향은 작습니다.

Q. 히스토그램과 상자그림은 뭐가 다른가요?

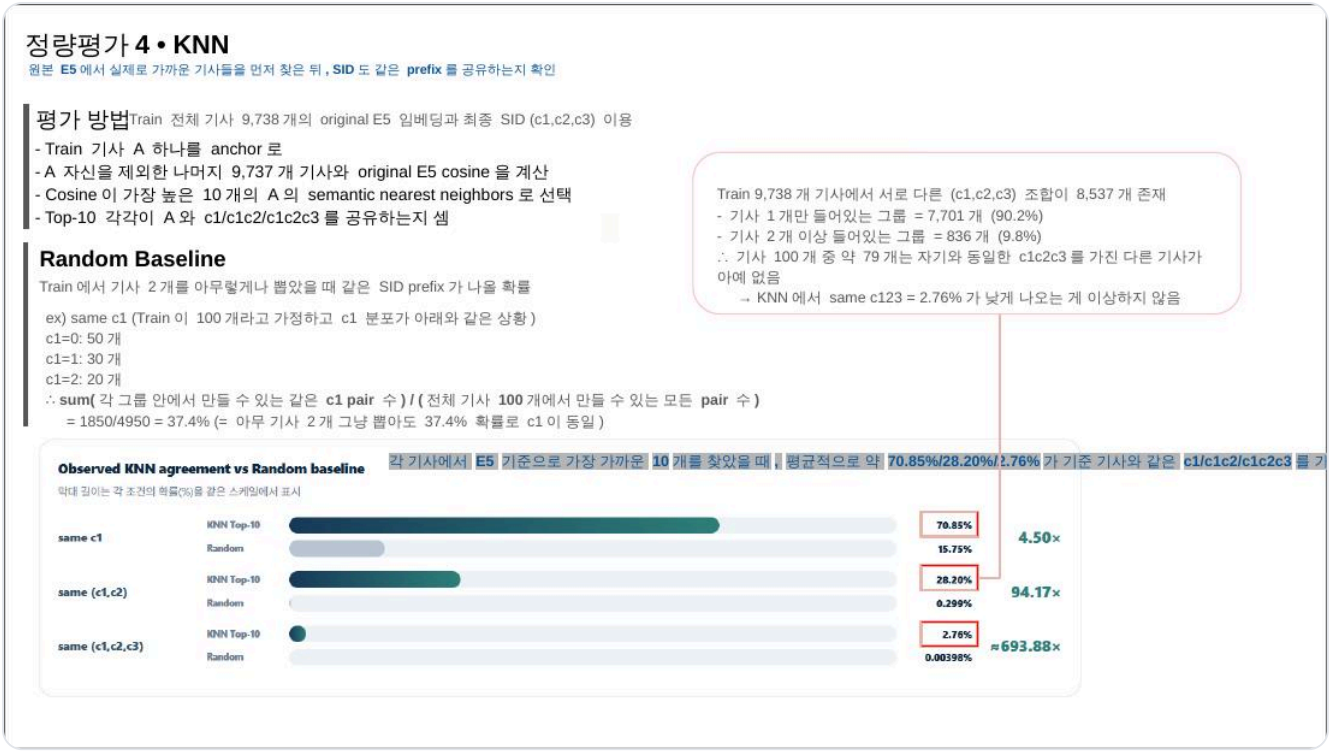
A. 둘 다 같은 기사별 점수입니다. 히스토그램은 분포 모양(어디에 몰렸나), 상자그림은 요약 숫자(중앙값 · 가운데 50%)를 보여 줍니다. 막대 높이 차이는 기사 수 차이입니다.

슬라이드에서 고칠 점

- 오른쪽 "결과 분석" 마지막 줄("돌아옴")이 슬라이드 밖으로 잘립니다.
- validation이 낮은 이유와 checkpoint 선택 주의점을 한 줄 넣으면 질문을 미리 막을 수 있습니다.

12장 · 정량평가4 · KNN

SID 품질 정량



11/05 발표 슬라이드 12

> 방향을 뒤집어 봐도 성립합니다. 원본 E5에서 가장 가까운 이웃 10개 중 **약 7개가 같은 c1, 3개가 같은 c1c2**를 갖고, 무작위의 4.5배 · 94배입니다.

발표 스크립트 (이렇게 말하면 됩니다)

앞 장들은 “SID가 같으면 가까운가”였고, 여기는 “원래 가까운 기사끼리 SID도 같은가”입니다.

train 기사 9,738개마다 나머지 9,737개와 E5 cosine을 계산해 가장 가까운 10개를 이웃으로 잡고, 이웃이 같은 c1 · c1c2 · c1c2c3를 가졌는지 셉니다. 9,738 × 10 = 97,380쌍에서 비율을 냅니다.

숫자만으로는 높은지 알 수 없어서, 아무 기사나 두 개 뽑았을 때 같은 prefix일 확률(무작위 기준)과 비교했습니다. c1이 70.85% 대 15.75%, c1c2가 28.20% 대 0.30%, c1c2c3가 2.76% 대 0.004%입니다.

c1c2c3 2.76%가 낮아 보이지만 train 기사의 79%는 같은 c1c2c3를 가진 다른 기사가 아예 없어서, 이웃을 완벽하게 골라도 최대 3.86%입니다. 최대치의 72%를 달성한 셈입니다.

숫자 읽는 법

기준	E5 이웃 10개 중 일치	무작위 기준	배수	이론상 최대 (달성률)
같은 c1	70.85%	15.75%	4.5×	99.8% (71%)
같은 c1c2	28.20%	0.299%	94×	85.2% (33%)
같은 c1c2c3	2.76%	0.004%	694×	3.86% (72%)

예상 질문과 답

Q. 무작위 기준은 왜 필요한가요?

A. 기사가 큰 카테고리에 몰려 있으면 이웃을 아무렇게나 골라도 같은 c1이 자주 나옵니다. 그래서 "SID와 이웃이 아무 관계 없을 때"의 숫자를 먼저 구해 비교합니다. 그룹마다 그 안에서 만들 수 있는 쌍 수를 더해 전체 쌍 수로 나눕니다.

Q. 새 기사(validation)에서도 그런가요?

A. 네. validation 3,122개 안에서 같은 방법으로 보면 c1 65.41%, c1c2 21.04%이고, 같은 크기(3,122개)로 뽑은 train 표본은 65.41%, 22.37%입니다. 새 기사에서도 이웃 보존 성질이 유지됩니다.

Q. E5는 RQ-VAE의 입력인데 자기 확인 아닌가요?

A. 맞습니다. 이 실험은 "입력 공간의 이웃 구조를 SID가 얼마나 보존하나"를 봅니다. 입력에 없던 본문으로 확인한 9장보다 근거로서는 약하고, 둘을 함께 봐야 합니다.

Q. c1이 30% 어긋나는 이유는?

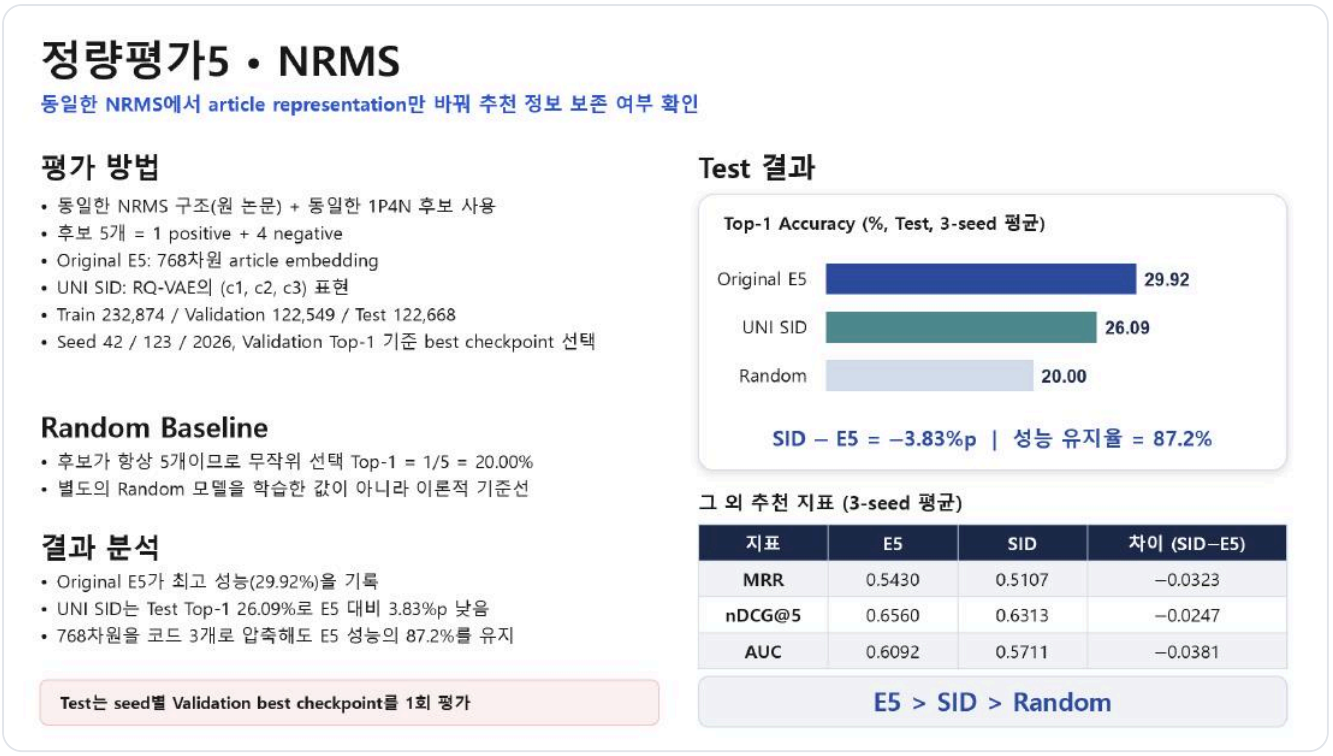
A. c1은 학습한 코드가 아니라 편집 카테고리로 고정한 값이라, E5가 보기엔 비슷해도 편집부가 다른 카테고리로 분류한 기사입니다. 설계의 대가입니다.

슬라이드에서 고칠 점

- 차트 위 설명 문장("각 기사에서 E5 기준으로 … 를 가진다")이 슬라이드 오른쪽 밖으로 잘립니다.
- 이론상 최대(3.86%)와 validation 결과(65% / 21%)를 한 줄씩 넣으면 "2.76%는 낮다", "새 기사는?" 질문에 바로 답할 수 있습니다.

13장 · 정량평가5 · NRMS

추천 성능 Test



11/05 발표 슬라이드 13

> 표준 추천 모델(NRMS)에서 기사 입력만 E5 → SID로 바꾸면 Top-1이 29.92% → 26.09%, **E5 성능의 87.2%가 남습니다.**

발표 스크립트 (이렇게 말하면 됩니다)

SID 자체의 품질 다음으로, 추천에 필요한 정보가 남는지 봤습니다. NRMS는 원 논문 구조 그대로(정민교 님 코드)이고, 기사를 벡터로 바꾸는 입구만 바꿨습니다.

E5 모드는 768차원 E5를 300차원으로 줄이는 Linear 층 하나를, SID 모드는 c1 · c2 · c3 번호마다 붙는 300차원 벡터 표 세 개를 새로 학습해 더합니다. 사용자 인코더(self-attention 20 head + additive attention)와 점수(내적), 데이터, seed는 모두 같습니다.

test 3-seed 평균으로 E5 29.92%, SID 26.09%입니다. 768차원을 코드 3개로 줄였는데도 성능의 87.2%가 남고, MRR · nDCG · AUC도 같은 방향입니다.

숫자 읽는 법

지표	E5	SID	차이
Top-1	29.92%	26.09%	−3.83%p (87.2%)
MRR	0.5430	0.5107	−0.0323
nDCG@5	0.6560	0.6313	−0.0247
AUC	0.6092	0.5711	−0.0381

NRMS는 어떻게 동작하나

읽은 기사 최근 50개 + 후보 5개 → ① 기사 인코더: 기사마다 300차원 (모드마다 다름) →

② 사용자 인코더: 읽은 기사들 → 사용자 벡터 1개 → ③ 점수 = 사용자 · 후보 (내적) 5개 →

1등 = 추천

단계	하는 일	모드별 차이
입력	1P4N 한 줄: 읽은 기사 ID(최근 50개, 모자라면 빈칸 + mask), 후보 5개 ID, 정답 표시. 기사 ID는 E5 파일 줄 번호(`embedding_row`)로 바꿔 표에서 값을 꺼냄	같음
① 기사 인코더	기사 하나 → 300차원 벡터. 읽은 기사와 후보 기사에 **같은 인코더**를 씀	**E5**: 768 → Linear → 300 (Linear만 학습, E5는 고정) **SID**: E1[c1] + E2[c2] + E3[c3] (번호마다 300차원 표를 새로 학습, RQ-VAE 코드북 아님) **Title**: 제목 단어 → 단어끼리 self-attention → 가중 평균 (원 논문)
② 사용자 인코더	읽은 기사 벡터끼리 self-attention(20 head)으로 서로 참고 → additive attention(300 → 200 → 1)으로 기사마다 중요도를 매겨 가중 평균 → 사용자 벡터. 빈칸은 mask로 무시	같음
③ 점수 · 출력	후보 5개 각각 사용자 벡터와 내적 → 점수 5개가 모델 출력. 1등이 정답이면 Top-1 정답	같음
학습	loss = −log softmax(점수)[정답] (cross-entropy). 문제 64개 평균으로 한 번 고침	같음

실제 예시로 따라가기

(실제) = 데이터에서 꺼낸 기사 · SID · 행 번호 · E5 값, **(설명용)** = 학습된 가중치가 필요한 값(학습된 checkpoint는 AWS에만 있음). "이 사용자가 이 기사들을 읽었다"는 상황은 설명용으로 만든 것입니다.

0. 상황

	기사 (실제)	SID	행 번호
읽음 h1	총리, 보고서 발언	12-22-288	8458
읽음 h2	투르 드 프랑스 중 전설이 빙에고르에게 경고	21-61-470	7499
읽음 h3	빙에고르, 폭설로 경기 중단	21-117-485	11057
후보 c1	해리 왕자 DNA 검사	22-105-2	17
후보 c2 ✅	빙에고르의 정상까지 (실제 클릭)	21-117-147	7532
후보 c3	브륀비 팬, 숨진 FCK 팬 조롱	21-88-469	1077
후보 c4	위기 테크 은행 500명 해고	15-29-449	19053
후보 c5	Side 9 인물 갤러리	20-122-349	631

1. 입력 만들기 — 기사 ID를 사전(`article_id2row`)으로 행 번호로 바꿉니다.

```
읽은 기사 50칸 = [8458, 7499, 11057, 빈칸, …, 빈칸]   mask = [1, 1, 1, 0, …, 0]
후보 5칸       = [17, 7532, 1077, 19053, 631]          정답 = 2번째 칸
```

2. 기사 인코더 — 읽은 기사 3개와 후보 5개, 8개 모두에 같은 방법으로 300차원 벡터를 만듭니다. 모드마다 여기만 다릅니다.

```
SID 모드
h3 (21-117-485) → E1[21] + E2[117] + E3[485]
c2 (21-117-147) → E1[21] + E2[117] + E3[147]   ← h3와 두 조각이 같음
c3 (21-88-469)  → E1[21] + E2[88]  + E3[469]   ← c1만 같음
c4 (15-29-449)  → E1[15] + E2[29]  + E3[449]   ← 겹치는 것 없음

E5 모드
h3의 E5 (실제) = [0.008, 0.032, −0.046, 0.012, …] (768개) → Linear 768→300 → 300개
```

- E1 · E2 · E3는 번호마다 300차원 벡터가 들어 있는 표로, NRMS가 처음부터 새로 학습합니다(RQ-VAE 코드북 아님). 코드에서는 0번을 빈칸 전용으로 두려고 번호에 1을 더해 씁니다(21 → 22). 의미는 같습니다.
- h3와 정답 c2는 "스포츠(21) · 자전거 경주(117)" 조각을 처음부터 공유합니다.
- E5만으로도 정답이 가장 가깝습니다(실제 계산). 읽은 기사 3개 평균과의 원본 E5 cosine: c2 **0.920**, c3 0.857, c4 0.857, c1 0.856, c5 0.844.

3. 사용자 인코더 (모든 모드 같음)

- (a) self-attention 20 head: 읽은 기사끼리 서로 참고합니다. 실제 E5 cosine도 h2–h3 0.864로 가장 가깝고 h1–h2 0.796, h1–h3 0.777이라, 빙에고르 기사 h2 · h3는 서로를 많이 참고하고 총리 기사 h1은 따로 남습니다. 빈칸 47개는 mask로 빼고, 20 head는 300차원을 15차원씩 20조각으로 나눠 20가지 관점에서 동시에 참고하는 것입니다.

- (b) additive attention (설명용): 기사마다 중요도를 매겨 가중 평균합니다. h1 0.15 · h2 0.41 · h3 0.43 · 빈칸 0 → 사용자 벡터 u = 0.15·h1′ + 0.41·h2′ + 0.43·h3′ = "자전거 · 빙에고르에 관심 많은 사용자".

4. 점수 = 출력 (모든 모드 같음, 설명용)

후보	점수 u · 후보	확률 (softmax)
c2 빙에고르 정상 ✅	4.1	0.82
c3 브뢴비 팬 (같은 스포츠)	2.3	0.13
c1 해리 왕자	0.4	0.02
c5 Side 9	0.3	0.02
c4 테크 은행	0.2	0.02

1등이 c2이고 실제 클릭도 c2이므로 Top-1 정답입니다. c3가 2등인 것은 SID 모드 기준 c1(스포츠) 조각을 공유하기 때문입니다.

5. 학습

- loss = −log P(정답) = −log 0.82 = 0.20 (설명용). 이 loss가 줄어드는 방향으로 SID 모드는 E1 · E2 · E3 표와 attention을, E5 모드는 Linear와 attention을 고칩니다.
- 문제 64개 평균 loss로 한 번 고치고, train 232,874문제를 한 바퀴 돌면 1 epoch, 최대 5 epoch.
- 매 epoch 끝에 validation half 122,549문제로 Top-1을 재 가장 좋은 epoch을 저장(SID 2 · 1 · 3, E5 5 · 5 · 4)하고, 마지막에 test 122,668문제로 한 번 채점 → seed 3개 평균 **E5 29.92%, SID 26.09%**.

```
읽은 기사 3개 + 후보 5개 (행 번호)
  → [기사 인코더] 기사마다 300차원   ← SID: E1+E2+E3 / E5: Linear(768→300)
  → [사용자 인코더] 읽은 기사끼리 참고(20 head) → 중요도 가중 평균 → u
  → [점수] u · 후보 5개 → c2가 1등 → 정답
  → [학습] −log P(c2)를 줄이도록 수정
```

SID 모드가 맞히는 이유 · 읽은 기사와 정답이 c1 · c2 조각을 공유하기 때문입니다. SID를 무작위로 섞으면 이 공유가 사라져 20.71%(무작위 수준)로 떨어집니다.

학습 설정과 선택

항목	값
코드	정민교 님 MIND NRMS 코드(원 논문 구조) 그대로, EB-NeRD 데이터 읽는 부분만 수정
구조	300차원 · self-attention 20 head · additive attention 200 · dropout 0.2
학습	Adam lr 1e-3 · batch 64 · **최대 5 epoch** (모든 모드 같음) · history 최근 50개
데이터	train 232,874 학습 / validation half 122,549로 epoch 선택 / test 122,668로 1회 보고
설정 선택	탐색 없음 (원 논문 설정 그대로)
checkpoint 선택	validation Top-1이 가장 높은 epoch
seed	42 · 123 · 2026, 3개 평균
best epoch	E5 5 · 5 · 4 (끝까지 오르는 중) / SID 2 · 1 · 3 (초반 최고 후 하락) / Title 2 · 5 · 3 / 섞은 SID 1 · 1 · 1

E5와 SID 차이를 읽을 때 · E5는 5 epoch에서도 오르는 중이었으므로 더 학습하면 차이(−3.83%p)가 더 벌어질 수 있습니다. 87.2%는 "5 epoch 고정 조건에서"입니다. SID는 1~3 epoch에서 최고라 더 학습해도 오르기 어렵습니다. 자세한 단계별 예시(실제 기사 · 행 번호 · 4차원 계산)는 `reports/1008_정민지파트_해설.html` 9장에 있습니다.

예상 질문과 답

Q. SID 번호에 정말 의미가 있나요? 그냥 번호 개수 효과 아닌가요?

A. 기사끼리 SID를 무작위로 섞은 대조군은 20.71%로 무작위 수준입니다. 번호 개수 · 그룹 크기는 같고 "어느 기사에 어떤 번호가 붙었나"만 다른데 +5.38%p 차이가 납니다. (슬라이드에는 뺐지만 답변용으로 기억해 두세요.)

Q. 제목으로 학습한 원래 NRMS와 비교하면요?

A. 제목 단어를 직접 읽는 원조 NRMS가 26.06%로 SID(26.09%)와 같은 수준입니다.

Q. SID 모드에서 RQ-VAE 코드북 벡터를 쓰나요?

A. 아닙니다. NRMS는 SID 번호만 받고 번호마다 붙는 벡터를 처음부터 새로 학습합니다. 그래서 "SID 번호 체계가 추천에 쓸모 있나"를 보는 실험입니다.

Q. Random 20%는 학습한 모델인가요?

A. 아닙니다. 후보가 5개라 아무렇게나 고르면 1/5이라는 이론값입니다.

Q. E5가 SID보다 높은 건 당연한가요?

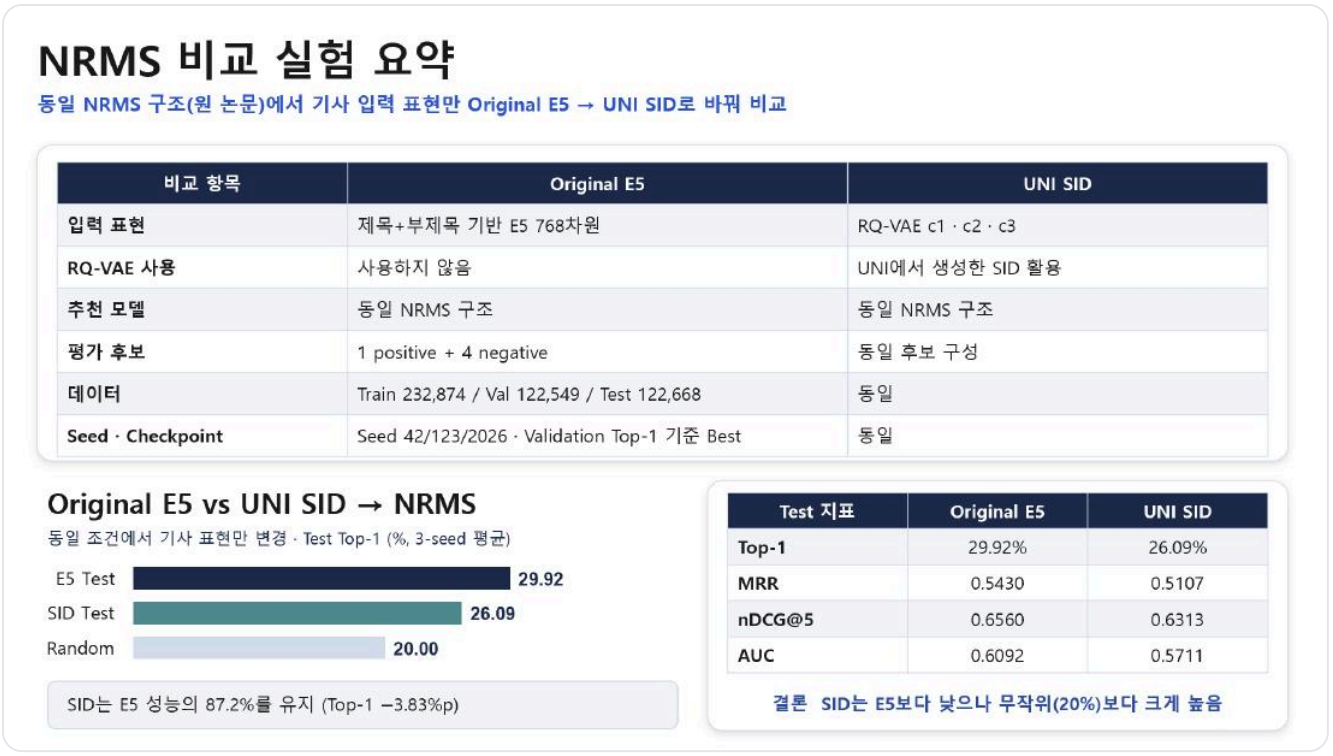
A. 네. SID는 E5를 코드 3개로 압축한 것이라 정보가 원본을 넘을 수 없습니다. c1c2c3가 같은 24%의 기사는 SID 모드에서 똑같은 벡터가 되고, 새 기사의 c3 번호 벡터는 train 클릭으로 충분히 학습되지 않습니다. 실험 목적은 이기는지가 아니라 얼마나 남는지(87.2%)입니다.

Q. SID도 최대 5 epoch인가요?

A. 네, 모든 모드가 최대 5 epoch입니다. SID는 2 · 1 · 3 epoch에서 최고였고, E5는 5 · 5 · 4로 끝까지 오르는 중이었습니다.

14장 · NRMS 비교 실험 요약

추천 성능



11/05 발표 슬라이드 14

> 13장의 설정과 결과를 표 하나로 요약한 장입니다.

발표 스크립트 (이렇게 말하면 됩니다)

비교 조건이 기사 입력 표현 하나뿐이라는 점을 강조합니다. 추천 모델, 후보 구성, 데이터(train 232,874 / validation 122,549 / test 122,668), seed와 checkpoint 기준이 모두 같습니다.

결론은 “SID는 E5보다 낮지만 무작위보다 크게 높고, E5 성능의 87.2%를 유지한다”입니다.

숫자 읽는 법

항목	Original E5	UNI SID
입력 표현	제목 + 부제 E5 768차원	RQ-VAE c1 · c2 · c3
RQ-VAE 사용	사용하지 않음	UNI에서 생성한 SID
추천 모델 · 후보 · 데이터 · seed	동일	동일
Test Top-1	29.92%	26.09%

예상 질문과 답

Q. E5도 RQ-VAE 입력이니 결국 같은 정보 아닌가요?

A. 맞습니다. SID는 E5를 압축한 것이라 E5가 상한입니다. 이 실험은 압축 후 얼마나 남는지(87.2%)를 재는 것입니다.

슬라이드에서 고칠 점

- 13장과 내용이 거의 같습니다. 발표 시간이 짧으면 둘 중 하나만 남기는 것을 권합니다(요약 표는 13장 왼쪽에 넣을 수 있음).

부록 · RQ-VAE vs Transformer V1 비교 (학습 · 선택 · 결과)

정리

> 둘 다 ① **설정 선택** + ② **epoch 선택** 두 층으로 골랐고, 둘 다 **validation만** 썼습니다. 차이는 ①을 RQ-VAE는 GPT가, Transformer는 코드 규칙이 판단했다는 점입니다.

1. 역할과 데이터

항목	RQ-VAE (UNI)	Transformer V1
하는 일	기사 E5 → SID (c1, c2, c3, c4) 만들기	사용자 기록 SID → 후보 5개 중 다음 클릭 고르기
입력	기사 하나의 E5 768차원 (제목 + 부제)	기록 최근 50개 기사의 c1 c2 c3 c4 토큰 + 후보 5개의 SID
출력	복원 벡터 $\hat{x}$ (768) + SID 번호	후보 5개 점수 S = L1 + L2 + L3
학습 데이터	train 기사 9,738개	1P4N train 232,874문제
validation	validation 기사 3,122개	validation half 122,549문제 (validation 주 앞쪽 절반)
test	없음 (클릭 예측 모델이 아님). SID 품질은 7~12장에서 따로 평가	122,668문제 (validation 주 뒤쪽 절반), 마지막에 1회

2. 모델과 학습

항목	RQ-VAE (UNI)	Transformer V1
구조	encoder 768→512→128 → 양자화 → decoder 128→512→768	T5식 encoder-decoder
크기	잠재 128 · c1 25(카테고리 고정) · Q2 128 · Q3 512	d_model 256 · head 8 · 2층 · d_ff 1024
핵심 방식	잔차 양자화 (STE, 정규화 없음, K-means 초기화)	level별 임베딩 · 출력 head, SEP 미사용, dropout 0
loss	복원 1 + codebook 0.25 + commitment 0.1 + uniqueness 0.05 (margin 0.5)	후보 5개 점수에 listwise cross-entropy
optimizer	lr 2e-4 · batch 256 · weight decay 0 (UNI 계열 gin 기준)	AdamW lr 5e-5 · batch 128 · weight decay 0
1 epoch 걸음 수	약 39 step	약 1,820 step
최대 epoch	1,283	30
평가 간격	10 epoch마다	매 epoch
early stopping	200 epoch 이후, 평가 20번 연속(약 200 epoch) 개선 없으면 정지 (개선 = 0.001 넘게 좋아짐)	5 epoch 연속 개선 없으면 정지
seed	42	42 (최종 상위 3개는 42 · 123 · 2026으로도 확인)

epoch 수가 크게 다른 이유: epoch은 데이터를 한 바퀴 도는 단위라 데이터 크기에 따라 길이가 다릅니다. RQ-VAE 180 epoch(≈ 7,020 step)과 Transformer 4 epoch(≈ 7,280 step)이 걸음 수로는 비슷합니다. RQ-VAE는 복원 loss가 천천히 오래 줄어 최대값을 넉넉히 잡았고, Transformer는 epoch 1에서 이미 validation 최고라 30이면 충분했습니다(84 run을 돌려야 하는 시간 제약도 있음).

3. 선택 방법

항목	RQ-VAE (UNI)	Transformer V1
① 설정 선택 지표	복원 loss ↓ · c123 충돌 ↓ · c2 사건 일관성 ↑ · Q2/Q3 사용률 ↑ · ΔC2/ΔC3 ↑	Top-1 → MRR → nDCG@5 → AUC → Preference Loss → Positive Probability 순서
① 판단 방식	단계별 결과표를 GPT가 종합 판단	코드에 고정된 규칙: 우선순위 + 허용 범위(Top-1 0.5%p 등), 동률이면 작은 모델
① 탐색 구조	코드북 → 잠재 · hidden → 양자화 → loss · 학습 단계, 마지막 weight decay 단계 001번 = WD-001 → uniqueness loss 추가 = UNI	6단계 × 단계마다 상위 2개 = 84 run, 마지막 상위 3개 seed 재확인
① 재현성	같은 판단이 나온다는 보장 없음	같은 결과표면 항상 같은 선택
② epoch 선택 지표	validation 복원 loss — 가장 낮은 epoch	validation Top-1 — 가장 높은 epoch
② 적용 범위	모든 run마다 자동 (`checkpoint_best_rec.pt`)	모든 run마다 자동 (`checkpoint_best`)
② 결과	epoch 180 (≈ 7,020 step)	epoch 1 (≈ 1,820 step)
best epoch 해석	천천히 오래 개선	첫 epoch 뒤 바로 과적합 (test 정답 대부분이 새 기사)
test를 선택에 썼나	안 씀	안 씀
주의	같은 run에 total loss 기준 best(epoch 20)도 저장되지만 쓰지 않음	탐색은 후보 순서를 섞기 전 데이터, 최종은 섞은 데이터로 재학습

```
RQ-VAE
설정 A 학습 → 10 epoch마다 validation 복원 loss → 최저 epoch 저장(best_rec) ┐
설정 B 학습 → 같은 방식 → best_rec                                          ├→ 각
best_rec로 SID → 5개 지표 표 → GPT 판단 → 다음 단계
   … 단계 반복 … → 최종(UNI) best_rec = epoch 180 → 최종 SID

Transformer V1
설정 8개 학습 → 매 epoch validation Top-1 → 최고 epoch 저장 → 규칙으로 비교 → 상위 2개
   … 6단계 84 run … → 상위 3개 seed 확인 → 최종 설정을 섞은 데이터로 재학습 → best epoch
1 → test 1회
```

4. 최종 결과

항목	RQ-VAE (UNI)	Transformer V1
핵심 결과	c123 충돌 24.39% (WD-001 31.56%)	test Top-1 27.07% (무작위 20%)
그 밖의 지표	validation 복원 loss 0.1525 · c2 사건 일관성 74.15% · Q3 512/512 · max c4 16	MRR 0.518 · nDCG@5 0.637 · AUC 0.579 · validation Top-1 27.06%
지표 기준 데이터	복원 loss는 validation 3,122개 / 충돌 · 일관성 · 사용률은 전체 12,860개	모두 test 122,668문제
후처리 개선	—	level 가중치 (1, 0.5, 0.1), validation에서 선택 → test 29.46%
코드 위치	`RQVAE/train_rqvae.py` (Colab에서 학습)	`minji1217/sid_project-transformer` 의 `Transformer/` (탐색: `sweep/`)

확인 필요 · RQ-VAE 설정 탐색의 정확한 단계 수 · 단계별 후보값 · GPT 판단 근거는 기록이 없습니다. 발표에서 물으면: "단계마다 다섯 지표를 표로 정리해 종합 판단했고 판단 보조로 LLM을 활용했습니다. 각 설정의 checkpoint는 validation 복원 loss 최저 epoch으로 자동 선택했습니다. 판단 규칙을 명시적으로 정해 재검증하는 것은 다음 과제입니다."

공통 예상 질문

Q. test는 어떻게 만들었나요?

A. EB-NeRD는 정답이 공개된 test가 없어서 validation 주의 클릭 기록을 시간순으로 반으로 나눴습니다. 앞쪽 절반(122,549문제)은 checkpoint · 설정 선택에만, 뒤쪽 절반(122,668문제)은 성적 보고에만 씁니다. 과거로 고르고 미래로 평가하는 구조입니다.

Q. 1P4N이 뭔가요?

A. 노출 한 번에서 클릭한 기사 1개(정답)와 보고도 클릭하지 않은 기사 4개(오답)로 문제 하나를 만든 것입니다. 정답과 c1c2c3가 같은 오답은 빼고, 오답이 4개가 안 되면 그 클릭은 버립니다. 무작위로 고르면 Top-1 20%입니다.

Q. 원래 설계와 지금 SID는 같은가요?

A. 두 군데가 다릅니다. ① 학습 중 c2를 사건 단위가 아니라 기사마다 고릅니다(사건 일관성 74%). ② 새 사건의 c2를 첫 기사가 아니라 사건 기사 전체 평균으로 정해, 신규 사건의 1.07%(32개)에서 추천 시점 이후 기사 정보가 들어갑니다. 여민 님이 첫 기사 기준으로 고치는 중입니다.

Q. 왜 seed가 장마다 다른가요?

A. NRMS(13장)는 seed 3개 평균, Transformer V1 · TIGER(5 · 6장)는 seed 42 한 번입니다. seed 하나짜리 결과는 1%p 안쪽 차이를 같은 수준으로 봅니다.

Q. 다음 단계는 무엇인가요?

A. ① c2를 원래 설계대로 사건 단위로(사건 일관성과 충돌의 균형) ② 새 사건 c2를 첫 기사 기준으로 ③ c3가 새 기사에 일반화되지 않는 문제(level 가중치 결과) 개선 ④ 추천 방식: 생성형 학습만으로는 실패하므로 후보 비교 학습과 결합.

근거: `reports/1008_정민지파트_해설.html`, `experiments/nrms_original_ebnerd_v2/`, `experiments/knn_e5_sid_v1/`, `experiments/uni_tiger_reference_v1/results_aws/`, `experiments/uni_v1_test_eval/`, 10/1 · 10/8 발표 자료